

Generador Web de Actividades y Contenido Dinámico

Descripción General

El objetivo de este proyecto es desarrollar una aplicación web basada en la arquitectura Cliente-Servidor que solicite información básica al usuario y devuelva una pantalla completamente nueva y personalizada. La aplicación constará de una página de inicio estática con un formulario tradicional. Al enviarse, el servidor propio (Backend) procesará la solicitud, se comunicará de forma asíncrona con dos proveedores de datos externos en internet (APIs públicas) para recolectar información relevante, y generará al vuelo una respuesta visual con un diseño limpio.

Este proyecto evalúa la integración de las tecnologías analizadas en clase: estructuración semántica con HTML, diseño con el modelo de caja de CSS, control de rutas con Express Router, aislamiento de configuraciones con variables de entorno (.env), y la implementación de lógica asíncrona y consumo de servicios externos mediante Promesas (async/await y fetch).

Especificaciones del Proyecto por Capas

1. Diseño de la Interfaz de Entrada (Frontend Estático)

La cara inicial de la aplicación se construirá exclusivamente utilizando estructuras de HTML y reglas de CSS estáticas.

- **Estructura Estática (HTML):** Crear un archivo de inicio en la raíz de la carpeta pública del servidor. Debe contener un encabezado principal con el nombre de la aplicación y un formulario de ingreso de datos (<form>). El formulario debe solicitar obligatoriamente el **Nombre** y el **Apellido** del usuario mediante campos de texto individuales (<input>). Este formulario se debe configurar de manera nativa para que su acción (action) apunte a la ruta de procesamiento del servidor local y utilice el método de transferencia de datos adecuado para formularios seguros (POST).
- **Estilos y Modelo de Caja (CSS):** Aplicar reglas visuales a los elementos del formulario. Cada campo de texto y el botón de envío deben contar con propiedades de borde, márgenes de separación y espaciados internos (rellenos) que garanticen un diseño ordenado, centrado y legible en la pantalla del navegador.

- **Restricción:** Esta página de inicio no debe contener ninguna línea de código JavaScript del lado del cliente. Toda la automatización del viaje de datos la realizará el navegador web de forma nativa a través del formulario.

2. Configuración del Servidor y Control de Entorno (Backend)

El núcleo de la aplicación se gestionará en el servidor local utilizando el entorno de Node.js.

- **Inicialización y Dependencias:** Configurar el entorno de ejecución instalando los módulos necesarios para levantar un servicio web interactivo y gestionar datos sensibles de configuración.
- **Procesamiento de Formularios:** Habilitar de forma obligatoria en el archivo principal del servidor el intermediario (middleware) nativo que faculta al sistema para leer e interpretar los datos crudos que viajan desde los formularios HTML.
- **Seguridad y Entorno:** Implementar un archivo oculto de variables de entorno para declarar y resguardar el puerto de escucha del servidor. Este archivo de configuración debe quedar explícitamente excluido del sistema de control de versiones.

3. Modularización de Rutas y Lógica Asíncrona (Patrón MVC)

Para mantener un código organizado y profesional, se aplicará la separación de responsabilidades en la gestión de las direcciones web del servidor.

- **Estructura del Enrutador:** Crear un módulo independiente encargado exclusivamente de las rutas de procesamiento de datos. Este enrutador debe conectarse bajo un prefijo común al servidor principal.
- **Captura de Datos:** Programar la ruta receptora para que atienda la solicitud del formulario. La función debe ser capaz de extraer el nombre y el apellido digitados por el usuario directamente desde el cuerpo de la petición.
- **Consumo Concurrente de Proveedores Externos:** Una vez capturados los datos del usuario, la función del servidor (que debe ser asíncrona) realizará de manera consecutiva dos peticiones web externas utilizando la herramienta de consultas nativa de Node.js:
 1. **Consulta de Actividad:** Conectarse a la API pública de actividades aleatorias para extraer el objeto informativo que contiene la sugerencia del día.
 2. **Consulta de Ilustración:** Conectarse a la API pública de imágenes de animales para extraer el objeto informativo que contiene el enlace web de la fotografía.
- **Extracción de Datos Precisos:** Utilizando la técnica de acceso por propiedad (notación de punto), los estudiantes deberán inspeccionar la estructura de la respuesta de cada API para aislar únicamente el texto de la actividad y la dirección URL de la imagen, descartando el resto de los datos enviados por los proveedores.

4. Generación de Respuesta Dinámica (Backend)

El servidor no responderá con datos planos en formato JSON, sino que construirá el código visual de la nueva pantalla que verá el usuario.

- **Construcción del Contenido:** Utilizando el mecanismo de envío de respuestas del servidor, redactar una estructura completa de documento HTML. Esta estructura debe fusionar de forma dinámica las variables locales con los datos de internet:
 - Debe incluir un saludo personalizado utilizando el nombre y apellido extraídos del formulario.
 - Debe desplegar dentro de un bloque de texto destacado la frase de la actividad aleatoria obtenida de la primera API.
 - Debe incorporar una etiqueta de imagen apuntando dinámicamente su origen a la URL de la fotografía obtenida de la segunda API, forzando dimensiones controladas en sus atributos de ancho y alto para evitar que la imagen desborde la pantalla.
 - **Estilos Embebidos de Acomodo:** La respuesta generada por el servidor debe incluir reglas de estilo básicas para que la caja de resultados, el saludo, la actividad y la imagen se muestren como una tarjeta unificada y centrada verticalmente en el navegador.
-

5. Proveedor 1: Propuesta de Actividad Dinámica

- **URL del Endpoint:** <https://bored-api.appbrewery.com/random>
- **Estructura del JSON de respuesta (Ejemplo real):**

```
{  
  "activity": "Learn how to whistle with your fingers",  
  "type": "education",  
  "participants": 1,  
  "price": 0,  
  "link": "",  
  "key": "2790226"  
}
```

- **Propiedad a aislar:** `.activity` (Tipo: Cadena de texto).

5. Proveedor 2: Proveedor 2: Ilustración del Entorno (Imágenes de Animales)

- **URL del Endpoint:** <https://dog.ceo/api/breeds/image/random>
- **Estructura del JSON de respuesta oficial:**

```
{  
  "message": "https://images.dog.ceo/breeds/deerhound-scottish/n0  
  "status": "success"  
}
```

- **Propiedad a extraer:** .message (Tipo: Cadena de texto que contiene la URL directa de la imagen en formato JPG).

Cronograma y Entregas Esperadas

Día 1: Arquitectura del Servidor y Conexión Externa (4 Horas)

El objetivo de la primera sesión es el desarrollo e integración de la lógica del Backend. Al finalizar este bloque de tiempo, el estudiante debe entregar un servidor Node.js completamente configurado, seguro con sus variables de entorno y modularizado con Express Router. La ruta receptora debe ser capaz de simular la llegada de información, realizar la comunicación asíncrona exitosa con ambas APIs externas mediante el uso de promesas, y demostrar en la terminal del sistema que logra aislar correctamente el texto de la actividad y la URL de la fotografía.

Día 2: Diseño de Entrada y Despliegue Dinámico (2 Horas)

El objetivo de la segunda sesión es la maquetación visual y el cierre del ciclo de datos. Al concluir el proyecto, el estudiante debe entregar la interfaz de inicio con el formulario estilizado bajo el modelo de caja de CSS. El sistema completo debe ser funcional: al llenar el formulario y presionar el botón de envío, el usuario debe experimentar de forma transparente el viaje hacia la nueva página generada por el servidor, visualizando su saludo personalizado, el titular de la actividad del día y la fotografía correspondiente en perfecto orden.

Rúbrica de Evaluación del Proyecto Integrador

La evaluación del proyecto se realizará con base en una escala de **100 puntos totales**, distribuidos en los siguientes criterios técnicos y operativos:

Criterio de Evaluación	Excelente (100%)	Debe Mejorar (50%)	Insuficiente (0%)	Puntaje
1. Entorno y Configuración Base (Backend)	Inicializa correctamente el proyecto. Configura el archivo .env para resguardar el puerto, levanta el servidor	Levanta el servidor, pero omite el uso de variables de entorno para el puerto, u olvida incluir los middlewares	El servidor no levanta, no se estructuró el entorno básico de Node.js o el	15 pts

	Express con éxito y aplica obligatoriamente los middlewares para archivos estáticos y procesamiento de formularios (<code>urlencoded</code>). <i>Excluye el .env del repositorio.</i>	necesarios, provocando que el servidor no pueda leer los datos del formulario HTML.	archivo .env fue expuesto en el repositorio.	
2. Modularización y Enrutamiento (MVC)	Implementa con éxito el uso de <code>express.Router()</code> . Separa la lógica de las rutas en un archivo independiente y lo conecta correctamente al servidor principal bajo un prefijo común, respetando el patrón de arquitectura visto en clase.	Crea la ruta funcionalmente, pero no utiliza <code>express.Router()</code> para modularizar el código, dejando la lógica mezclada directamente en el archivo principal <code>app.js</code> .	No se crearon las rutas requeridas o el enrutador no logra conectarse con el servidor principal.	15 pts
3. Lógica Asíncrona y Consumo de APIs	Captura exitosamente los datos del formulario mediante <code>req.body</code> . Implementa funciones asíncronas con <code>async/await</code> y realiza de forma concurrente las dos peticiones <code>fetch</code> hacia los proveedores externos, manejando posibles fallos con bloques <code>try/catch</code> .	Realiza las peticiones externas, pero no logra capturar correctamente los datos de los inputs del formulario, o confunde la sintaxis de las promesas generando bloqueos en el hilo de ejecución.	No implementa la lógica asíncrona, no utiliza <code>fetch</code> o las consultas a las APIs externas fallan de forma constante sin manejo de errores.	25 pts
4. Extracción de Datos y Respuesta Dinámica	Utiliza con precisión la notación de punto para extraer exclusivamente la	Envía una respuesta HTML, pero los datos se muestran incompletos, no	El servidor no genera ninguna respuesta	25 pts

	actividad (<code>.activity</code>) y la URL de la imagen (<code>.message</code>). Construye y envía un documento HTML al vuelo con <code>res.send()</code> que fusiona los datos del formulario con los de las APIs en un diseño limpio.	logra aislar las propiedades correctas del JSON externo (muestra datos indefinidos) o carece del enlace para volver al inicio.	visual para el navegador o devuelve un formato JSON plano en lugar de la estructura de página solicitada.	
5. Frontend Estático y Modelo de Caja (HTML/CSS)	Maqueta la página de inicio de forma lineal y semántica sin usar JavaScript en el cliente. Configura el formulario nativo con <code>action</code> y <code>method="POST"</code>. Aplica los estilos básicos de CSS (borde, margen, relleno y esquinas redondeadas) de manera limpia.	El formulario envía los datos, pero carece de estructura semántica básica, o los elementos visuales se desbordan por no aplicar correctamente las propiedades de ancho, alto y modelo de caja en el CSS.	No entregó la interfaz gráfica, el formulario no se comunica con el backend debido a errores de configuración nativa o incluye código JavaScript restrictivo en el cliente.	20 pts